



## UNIDADE I

---

### Análise e Projeto de Sistemas

Profa. MSc. Cristiane Fidelix

# Capítulo 1 - Introdução à análise e projeto de sistemas

## Índice: 1ª parte

- Introdução: Sistema de Informação X Software
- Fundamentos da Engenharia de Software
- Marcos Históricos da Evolução do Software

# Capítulo 2 - Projeto de desenvolvimento

**Índice:** 2ª parte

- Processos de Desenvolvimento de Software
- Modelos de Processo de Software: Cascata X Modelos Ágeis (Scrum, XP)

## Capítulo 2 - Projeto de desenvolvimento

**Índice:** 3ª parte

- Continuação dos Modelos de Processo de Software: Incremental, Espiral e Prototipagem e Processo Unificado.

# Capítulo 2 - Projeto de desenvolvimento

**Índice:** 4ª parte

- Conceitos: Paradigmas da Orientação a Objetos
- Pilares da Orientação a Objetos: Objetos, Classe, Herança, Polimorfismo

# Introdução à análise e projeto de sistemas

- Neste capítulo, abordaremos os fundamentos essenciais da análise e projeto de sistemas de software, explorando a relevância na construção de soluções tecnológicas modernas.
- As etapas cruciais garantem que os sistemas desenvolvidos atendam efetivamente às necessidades do negócio, resultando em soluções robustas, flexíveis e sustentáveis.

# Fundamentos da engenharia de software

- A engenharia de software é o alicerce para a criação de sistemas que realmente resolvem problemas do mundo real.
- Desde sua origem, essa área passou por uma evolução, buscando sempre maior eficiência, qualidade e capacidade de adaptação.

# Evolução do software

- Com essa evolução, os sistemas de software ficaram necessariamente mais completos e complexos.
- Muito disso se deve à necessidade de se utilizar todos os recursos disponíveis, recursos estes que faltavam em outros momentos.
- Segundo Stallings (2003), a evolução dos computadores é evidenciada, ao longo dos anos, pelo aumento da capacidade dos processadores, da memória e da velocidade do mecanismo entrada/saída.



## Marcos históricos

- Essa evolução deve-se ao fato de que os componentes de um processador, também chamados de transistores, foram diminuindo em tamanho com o passar do tempo.
- Com essa diminuição, foi possível reduzir o espaço entre esses componentes.
- Diminuindo também a distância que uma corrente elétrica percorre entre dois ou mais transistores, obtendo um consequente ganho na velocidade de processamento.

# Marcos históricos

- Esses fatores geraram uma atmosfera positiva que culminaria, na década de 1980, com a massificação do uso de computadores por usuários comuns, o uso dos computadores pessoais.
- Após a massificação do uso de computadores, surgiram novos desafios. As máquinas passaram a ser operadas por usuários comuns. Não era mais necessário ser um especialista, os computadores passaram a ser mais fáceis de serem usados.

**Desafio posto:** como desenvolver sistemas que acompanhassem a tendência da massificação do uso?

# Marcos históricos

- Além do aumento na velocidade do processamento, a diminuição dos componentes gerou um espaço maior na pastilha dos processadores que, por sua vez, foram preenchidas por mais transistores, havendo, conseqüentemente, um novo ganho de velocidade.
- Com esses e muitos outros desafios, passamos a ter uma maior preocupação com a informação. A forma como a informação era trabalhada, trafegada, levada ao usuário e gerenciada passou a ter papel de fundamental importância.
- Surgiu então o conceito de sistemas de informação.

# Sistema de informação X software

## O que significa a palavra sistema?

- **Exemplo:** O que é o sistema imunológico de uma pessoa? De forma resumida e popular, o sistema imunológico biológico é a composição de diferentes tipos de células, cada qual com sua função, que atuam conjuntamente com o objetivo de manter a integridade do organismo que está sendo protegido.

# Sistema de informação X software

## E o que é um sistema de informação?

- **Exemplo:** Quando falamos em tecnologia de sistemas de informação, estamos falando de toda plataforma tecnológica que dá suporte ao processo de negócio, como infraestrutura de rede, servidores, computadores, sistemas operacionais e sistemas de software.

# Sistema de informação X software

## O que é um sistema de software?

- Um sistema de software é um conjunto de vários componentes que, quando executados, produzem um resultado previamente desejado.

# Sistema de informação X software

- Os componentes de software são desenvolvidos utilizando métodos e processos que estabelecem uma relação entre a necessidade do cliente e um código executado em uma máquina (Pressman, 2006).
- A engenharia de software tem por objetivo apoiar o desenvolvimento de software fornecendo técnicas para especificar, projetar, manter e gerir um sistema (Sommerville, 2010).
- Ela é baseada em três pilares: métodos, ferramentas e processos.

# Interatividade

Assinale a alternativa que indica corretamente os elementos que apoiam o desenvolvimento de software.

- a) A análise de software, os métodos e as ferramentas.
- b) Apenas as ferramentas, que dão suporte à execução das atividades descritas no método.
- c) Apenas os processos, que proveem o elo entre métodos e ferramentas, quando e onde fazer.
- d) Apenas os métodos, que são um conjunto de atividades para desenvolver um sistema de software.
- e) Os métodos, as ferramentas e os processos.



## Resposta

Assinale a alternativa que indica corretamente os elementos que apoiam o desenvolvimento de software.

- a) A análise de software, os métodos e as ferramentas.
- b) Apenas as ferramentas, que dão suporte à execução das atividades descritas no método.
- c) Apenas os processos, que proveem o elo entre métodos e ferramentas, quando e onde fazer.
- d) Apenas os métodos, que são um conjunto de atividades para desenvolver um sistema de software.
- e) Os métodos, as ferramentas e os processos.

# Capítulo 2 - Projeto de desenvolvimento

**Índice:** 2ª parte

- Modelos de Processo de Software
- Cascata
- Modelos Ágeis: Scrum e XP

# Metodologias tradicionais X ágeis

- Existem duas abordagens principais no desenvolvimento de sistemas: a tradicional, o modelo cascata; e o modelo ágil, exemplificado pelo Scrum e XP.
- Cada abordagem oferece metodologias específicas e diferentes estratégias na gestão do projeto de software.
- A escolha entre elas depende das características do projeto, tais como complexidade, estabilidade dos requisitos e necessidade de flexibilidade.

## Metodologias tradicionais – Modelo cascata

- As metodologias tradicionais, como o modelo cascata, propõem uma abordagem sequencial e estruturada. Assim também conhecido como modelo Waterfall, é um modelo de desenvolvimento de software sequencial e linear, composto por fases distintas e interconectadas.
- Nesse modelo, cada fase do desenvolvimento deve ser totalmente concluída antes de iniciar a seguinte, garantindo uma documentação detalhada e rigorosa.

## Metodologias tradicionais – Modelo cascata

- Uma desvantagem e característica do modelo cascata é sua baixa flexibilidade em alterações de requisitos.
- Mudanças realizadas após o início das etapas podem causar grandes retrabalhos e custos adicionais. Outra desvantagem dessa metodologia é o tempo prolongado até a entrega final do produto. Como todas as etapas devem ser completadas sequencialmente, os usuários só podem acessar o sistema após a conclusão de todo o processo.

# Metodologias tradicionais – Modelo cascata

## Exemplo:

- Um sistema ERP pode levar meses ou anos até ser completamente implantado e operacionalizado, dificultando respostas rápidas a necessidades emergentes.
- Um sistema de gestão de biblioteca que precise incorporar novas funcionalidades após a fase de implementação pode enfrentar dificuldades técnicas e aumento expressivo de custos.

# Metodologias ágeis – Scrum e XP

- As metodologias ágeis, como Scrum e XP, oferecem um desenvolvimento flexível, iterativo e incremental, adaptando-se rapidamente às mudanças.
- O desenvolvimento ocorre em ciclos curtos chamados sprints, proporcionando entregas frequentes de funcionalidades incrementais.
- Isso permite maior interação com os usuários e ajustes contínuos conforme o feedback recebido.

# Metodologias ágeis – Scrum e XP (Extreme Programming)

- A metodologia Scrum é ideal para organizar os ciclos curtos de entrega (sprints), planejar tarefas com clareza e revisar os incrementos com os stakeholders ao final de cada ciclo.
- A Extreme Programming possui boas práticas, como programação em pares, integração contínua e testes automatizados, que são valiosas para treinar os membros juniores e garantir a qualidade do código desde o início.
- É uma metodologia ágil para equipes pequenas e médias que desenvolvem software baseado em requisitos vagos e que se modificam rapidamente.



# Metodologias ágeis – Scrum e XP

- Uma das vantagens significativas dos métodos ágeis é o engajamento constante dos usuários e clientes.
- A comunicação frequente permite adaptações rápidas às necessidades identificadas pelos usuários durante o desenvolvimento.
- Outra vantagem fundamental das metodologias ágeis é a flexibilidade para alterações frequentes.

## Metodologias ágeis – Scrum e XP

- A possibilidade de ajustar ou acrescentar novos requisitos ao longo do projeto minimiza retrabalhos extensivos.
- Uma das desvantagens seria com relação a planejamento e prazos.
- As metodologias ágeis também têm desafios, como menor previsibilidade no planejamento de longo prazo.

## Metodologias ágeis – Scrum e XP

- A flexibilidade pode dificultar estimativas precisas de prazos e custos, especialmente com frequentes alterações de requisitos.
- Além disso, exigem alto envolvimento contínuo do cliente, o que pode ser um desafio se ele não estiver regularmente disponível para fornecer feedback necessário.

# Metodologias ágeis – Scrum e XP

## Exemplo:

- Startups conseguem rapidamente lançar e ajustar funcionalidades de aplicativos conforme o feedback dos usuários.
- Um sistema como o de gestão de biblioteca pode rapidamente incorporar novas funcionalidades, como reservas online ou notificações automáticas, adaptando-se às demandas dos usuários.

# Interatividade

Qual das alternativas abaixo descreve uma característica exclusiva da metodologia Scrum?

- a) As fases devem ser executadas sequencialmente, sem sobreposição.
- b) Mudanças são facilmente incorporadas durante o desenvolvimento.
- c) O projeto exige documentação detalhada antes do início das atividades.
- d) É ideal para projetos em que os requisitos são fixos e bem definidos desde o início.
- e) Cada fase do projeto precisa ser totalmente finalizada antes do início da próxima.

## Resposta

Qual das alternativas abaixo descreve uma característica exclusiva da metodologia Scrum?

- a) As fases devem ser executadas sequencialmente, sem sobreposição.
- b) Mudanças são facilmente incorporadas durante o desenvolvimento.**
- c) O projeto exige documentação detalhada antes do início das atividades.
- d) É ideal para projetos em que os requisitos são fixos e bem definidos desde o início.
- e) Cada fase do projeto precisa ser totalmente finalizada antes do início da próxima.

## Capítulo 2 - Projeto de desenvolvimento

**Índice:** 3ª parte

- Continuação dos Modelos de Processo de Software: Incremental, Espiral e Prototipagem e Processo Unificado.

# Modelo incremental

- A diferença é que no modelo incremental não é necessário que todos os requisitos estejam mapeados para se iniciar o ciclo de desenvolvimento.
- O ciclo é iniciado a partir de blocos de requisitos, e não a partir de todos os requisitos, como pregado no modelo clássico.
- A partir daí, a metodologia é a mesma, ou seja, cada atividade somente é iniciada depois que a predecessora é finalizada e validada.



# Modelo incremental

- No modelo iterativo, as atividades e a sequência de execução são exatamente as mesmas do modelo cascata, mas não é necessário que todos os requisitos estejam mapeados para se iniciar o ciclo de desenvolvimento.
- O ciclo é iniciado a partir de blocos de requisitos, e não a partir de todos os requisitos, como pregado no modelo clássico.

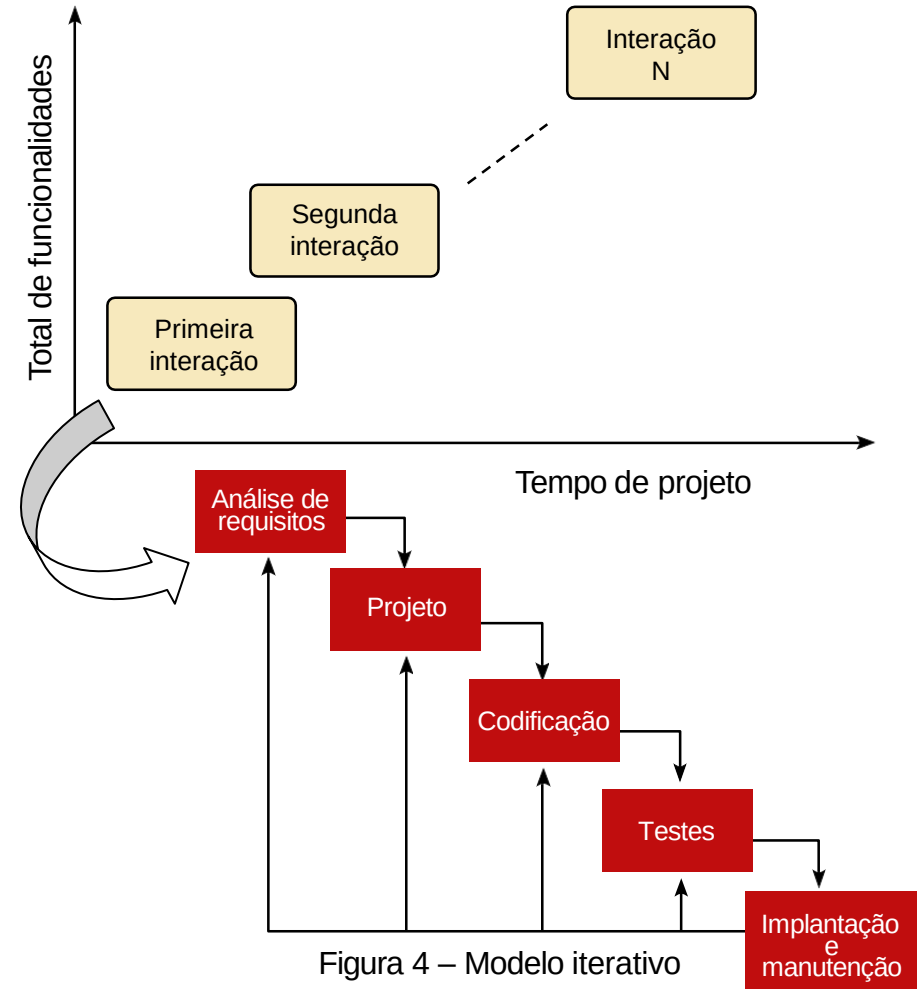


Figura 4 – Modelo iterativo

# Modelo incremental

- **Vantagens:** Um dos pontos positivos na adoção do modelo incremental é a redução do retrabalho.
- O número menor de requisitos reduz a chance de problemas na análise, projeto e construção.
- Ademais, com um bom planejamento de execução, é possível priorizar partes de requisitos críticos para o negócio, requisitos que gerem valor ao processo, abrindo a possibilidade da vantagem competitiva para a organização no mercado.

# Modelo espiral

- **O modelo espiral é composto de quatro fases:** Definição dos objetivos, Mitigação dos riscos, Desenvolvimento do produto e Planejamento da próxima fase.

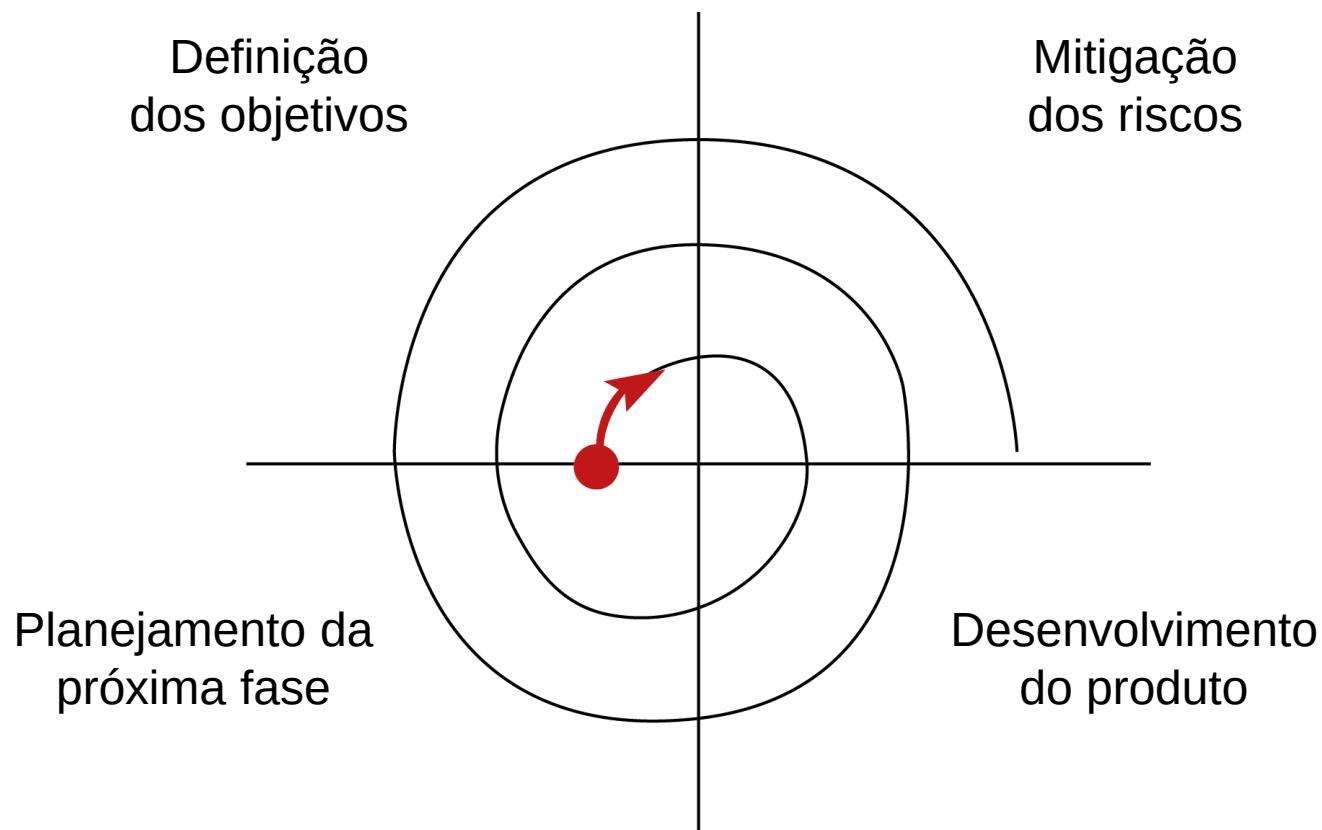


Figura – Modelo espiral

# Modelo espiral

- A figura anterior representa o modelo espiral proposto por Boehm (1988). Cada linha da espiral, em cada fase, corresponde a uma sequência de atividades.
- **Por exemplo:** na fase de desenvolvimento do produto, podemos ter elicitação de requisitos, documentação e validação dos requisitos.

# Modelo espiral

- A ênfase na diminuição do risco se dá pelo fato de que entre a definição do que precisa ser feito e o efetivo desenvolvimento, existe a fase de mitigação dos riscos.
- Nela, os riscos são identificados, mitigados e, a partir disso, são traçadas estratégias para minimizar os impactos para as fases seguintes.

# Prototipagem

- Segundo o dicionário, protótipo é um primeiro tipo, um primeiro exemplar.
- Em engenharia de software podemos pensar que o protótipo é a primeira versão de um sistema de software (Sommerville, 2011).
- Esse protótipo de software tem como objetivo a prova de conceitos. Verificar e demonstrar se os requisitos mapeados estão de acordo com a necessidade do usuário ou validar uma solução de projeto.

# Prototipagem

- É fato que a prototipagem é utilizada com o objetivo de antecipar mudanças que possam vir a ser mais custosas no desenvolvimento de um sistema de software.

**O modelo de prototipagem possui quatro atividades:**

- Planejamento, definição das funcionalidades, construção e validação.

# Prototipagem

- **Vantagens:** Protótipos de baixa fidelização foram eficientes e viáveis para demonstrar para os usuários as funcionalidades a serem implementadas pelo sistema.
- **Outro fato interessante observado foi a viabilidade econômica do protótipo:** barato na construção e na alteração.
- **Alguns dos bons resultados obtidos nesse projeto:** rapidez no processo de captação dos requisitos e antecipação dos problemas.



# Prototipagem

## Como sabemos quando usar a prototipação?

Pressman (2006) faz algumas observações bem interessantes:

- Se nosso usuário tem uma boa noção, mas não tem, no momento, uma visão detalhada do que precisa ser feito, é positivo que usemos a prototipação como um “primeiro passo”.

# Processo unificado - RUP

- Idealizado por Jacobson, Rumbaugh e Booch (1999), o processo unificado, também conhecido como UP (Unified Process), tem em seu fundamento a estrutura do modelo iterativo incremental.
- Uma variação importante do modelo de processo unificado, que é utilizada largamente pelas grandes organizações atualmente, é o RUP (Rational Unified Process) (Kruchten, 2003).

# Processo unificado - RUP

O RUP é um modelo de processos, desenvolvido pela Rational, que associou o modelo de processo unificado às melhores práticas da engenharia de software, que são:

- Desenvolva iterativamente, gerencie requisitos, use arquitetura de componentes, modele visualmente, verifique continuamente a qualidade, gerencie mudanças.
- As fases do RUP são as mesmas definidas no UP: iniciação, elaboração, construção e transição.

# Interatividade

Qual das alternativas abaixo corresponde ao modelo de desenvolvimento de software Prototipação?

- a) Definição das funcionalidades e construção.
- b) Definição das funcionalidades, construção e validação.
- c) Planejamento, definição das funcionalidades, construção e validação.
- d) Planejamento, construção e validação.
- e) Construção das funcionalidades e validação.

## Resposta

Qual das alternativas abaixo corresponde ao modelo de desenvolvimento de software Prototipação?

- a) Definição das funcionalidades e construção.
- b) Definição das funcionalidades, construção e validação.
- c) Planejamento, definição das funcionalidades, construção e validação.
- d) Planejamento, construção e validação.
- e) Construção das funcionalidades e validação.

## Capítulo 2 - Projeto de desenvolvimento

**Índice:** 4ª parte

- Paradigmas da orientação a objetos
- Pilares da orientação a objetos: Objetos, Classe, Herança, Polimorfismo

# Paradigma da orientação a objetos

- **O paradigma estruturado é baseado em dois elementos:** informações (dados) e processos. Nesse pensamento, os processos agem diretamente sobre os dados para a execução de uma determinada tarefa.
- **Exemplo: solução de um problema.**  
Como troco uma lâmpada?

# Paradigma da orientação a objetos

- Se pensarmos no paradigma da orientação a objetos, podemos interpretar que temos dois objetos no processo: cliente e terminal.
- Possuem características, comportamentos e executam determinadas ações.
- Em orientação a objetos, cada objeto também possui características, denominadas atributos, e a ação a ser executada por esse objeto, que tem o nome de método.



# Paradigma da orientação a objetos

O paradigma da orientação a objetos é baseado nos seguintes pilares:

- Objeto, classes, herança, polimorfismo e encapsulamento.
- **Exemplo:** O que temos em um campo de futebol?

# Conceito de orientação a objetos

**Objeto:** qualquer elemento do mundo real.

**Exemplo:** Aluno

- **Atributo:** Características?
- **Método:** Comportamento?

# Conceito de orientação a objetos

**Classe:** conjunto de objetos do mundo real.

**Exemplo:** Alunos

- **Atributo:** RA, nome, idade etc.
- **Método:** Estudar, fazer prova, entregar atividade etc.

# Conceito de orientação a objetos

- **Herança:** Assim como no mundo real, existem objetos que possuem características semelhantes, mas que estão em classes diferentes.

- **Exemplo:** classe Cliente

**Atributo:** ContaCorrente,  
NumeroAgencia, NumeroBanco

**Método:** efetuarSaque ()

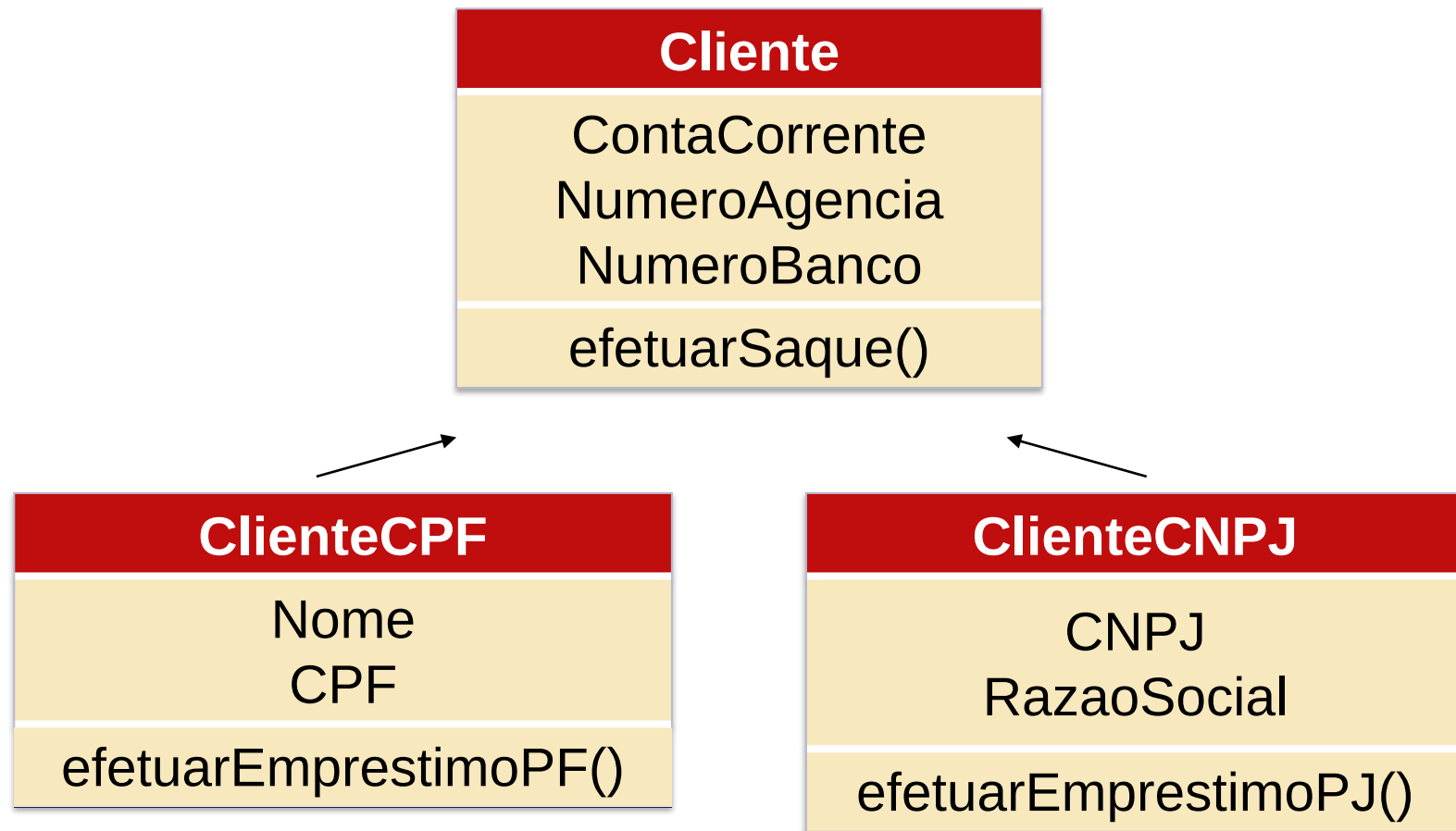
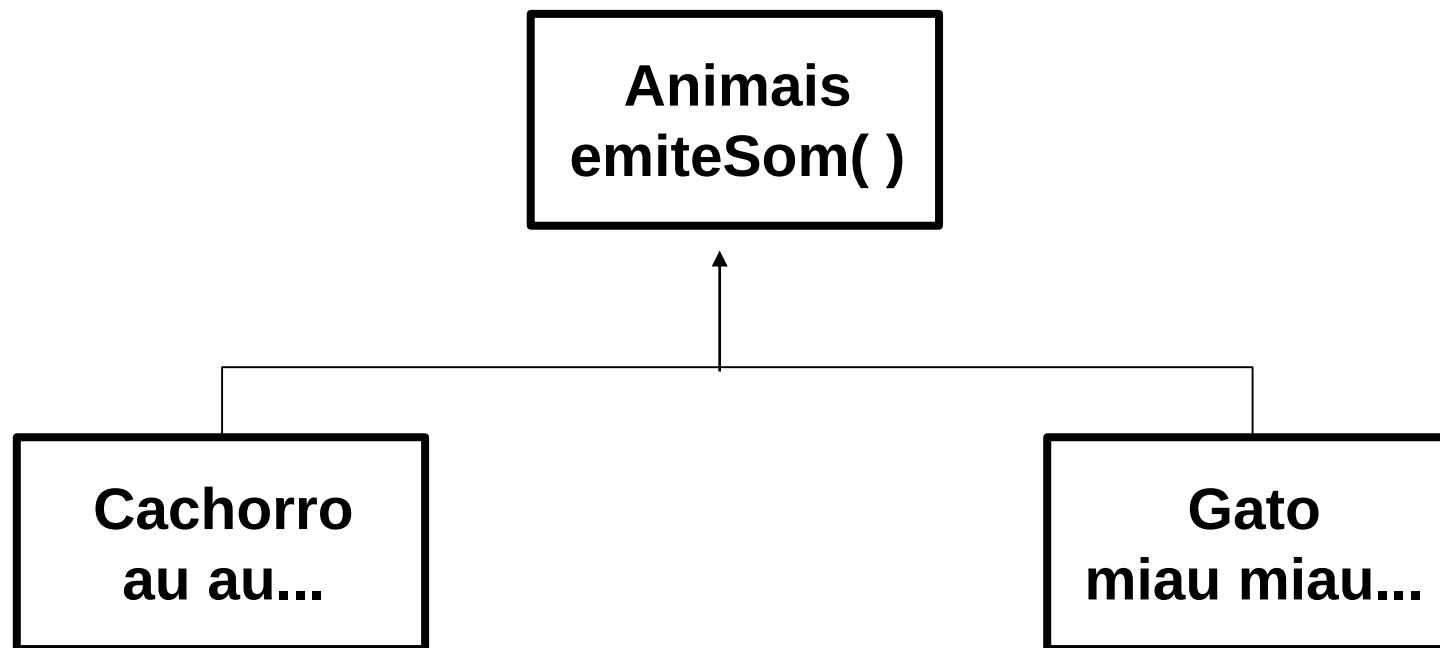


Figura – Exemplo de herança

# Conceito de orientação a objetos

- Polimorfismo é quando um objeto tem um comportamento diferente para a mesma ação.
- **Exemplo:** podemos fazer uma analogia aos animais, assim temos a classe: Animais



Fonte: autoria própria.

# Interatividade

Qual das alternativas abaixo corresponde a um atributo e a um método da classe Conta?

- a) numeroConta.
- b) pessoaFisica e pessoaJuridica.
- c) nomeCliente e numeroAgencia.
- d) emitirExtrato e transferirValor.
- e) numeroConta e emitirExtrato.

## Resposta

Qual das alternativas abaixo corresponde a um atributo e a um método da classe Conta?

- a) numeroConta.
- b) pessoaFisica e pessoaJuridica.
- c) nomeCliente e numeroAgencia.
- d) emitirExtrato e transferirValor.
- e) numeroConta e emitirExtrato.

# Referências

- BOEHM, B. W. A spiral model of software development and enhancement. *Computer*, California, v. 21, n. 5, p. 61-72, maio 1988.
- BOOCH, G.; JACOBSON, I.; RUMBAUGH, J. *UML - guia do usuário*. 2. ed. Rio de Janeiro: Campus, 2006.
- KRUCHTEN, P. *The rational unified process: an introduction*. Boston: Addison-Wesley, 2003.
- LARMAN, C. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao processo unificado*. 2. ed. Porto Alegre: Bookman, 2007.
- PRESSMAN, R. S. *Engenharia de software*. 6. ed. São Paulo: Mcgraw-Hill, 2006.
- PRESSMAN, R. S. *Engenharia de software*. 7. ed. São Paulo: McGraw-Hill, 2011.
- SCHACH, S. R. *Engenharia de software: os paradigmas clássicos orientados a objetos*. São Paulo: McGraw-Hill, 2009.
- SOMMERVILLE, I. *Engenharia de software*. São Paulo: Pearson, 2010.
- SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson, 2011.
- STALLINGS, W. *Arquitetura e organização de computadores*. São Paulo: Prentice Hall, 2003.



**ATÉ A PRÓXIMA!**